Assignment 3: Transformer is All You Need

Collin Gates

April 17, 2026

1 Introduction

The Transformer architecture has become the core building block behind modern large language models due to its ability to model long-range dependencies efficiently through self-attention. Unlike recurrent architectures, Transformers process entire sequences in parallel, enabling faster training and improved scalability.

In this assignment, we implemented a lightweight decoder-only Transformer model from scratch using PyTorch and trained it on the Tiny Shakespeare corpus for the next-token prediction task. The goal was to understand the role of token embeddings, positional encoding, masked self-attention, residual connections, and feed-forward networks in autoregressive language modeling.

Beyond implementation, the assignment also focused on analyzing model behavior through loss curves, attention heatmaps, and attention distance statistics in order to interpret how the model learns contextual dependencies in text.

2 Dataset and Preprocessing

2.1 Dataset

We used the Tiny Shakespeare corpus, a compact text dataset commonly used for character-level or token-level language modeling experiments. Despite its small size, the dataset contains enough linguistic structure to evaluate whether a Transformer can learn meaningful sequential patterns.

2.2 Tokenization

The raw text was tokenized using a Byte Pair Encoding (BPE) subword tokenizer trained from scratch on the corpus, with a vocabulary size of 400 tokens (within the 500-token limit). BPE starts from a byte-level vocabulary and iteratively merges the most frequent adjacent token pair, producing sub-word units that balance compression with linguistic coverage. Frequent Shakespearean morphemes such as `-ing`, `-eth`, and common function words emerged naturally as merged tokens.

Each token was mapped to an integer ID, which was later converted into dense embeddings via `nn.Embedding` before being processed by the Transformer layers.

2.3 Sequence Construction

The tokenized stream was divided into overlapping fixed-length sequences of 50 tokens with a stride of 25. For each training example:

- The input consisted of the first N tokens
- The target consisted of the same sequence shifted by one token

This setup allows the model to learn the conditional probability of the next token given all previous tokens.

2.4 Data Split

The processed sequences were divided into 80% training and 20% validation. The validation set was used to monitor generalization and compute perplexity. No held-out test set was required for this assignment.

3 Model Architecture

A lightweight decoder-only Transformer was implemented. Table 1 summarizes the full configuration.

Hyperparameter	**Value**
Transformer blocks L	2
Model dimension d	128
Attention heads H	4
Head dimension $d_h = d/H$	32
FFN inner dimension	512 ($4 \times d$)
Context window T	50 tokens
Vocabulary size V	400
Dropout	0.10
Total parameters	446,336

Table 1: Model architecture and hyperparameters.

The input token embedding weight matrix was tied to the final language model head, reducing parameter count and improving generalization.

3.1 Positional Encoding

Since self-attention is permutation-equivariant by design, positional information must be injected explicitly. We used fixed sinusoidal positional encodings from Vaswani et al. (2017):

$$\mathrm{PE}(pos,\, 2i) = \sin\Big(\frac{pos}{10000^{2i/d}}\Big), \qquad \mathrm{PE}(pos,\, 2i+1) = \cos\Big(\frac{pos}{10000^{2i/d}}\Big)$$

These encodings are added to the token embeddings before the first Transformer block. Varying frequencies across dimensions allow the model to represent relative positional offsets without any additional learnable parameters.

3.2 Masked Self-Attention

Self-attention computes pairwise interactions between all tokens in a sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

where the causal mask $M_{ij} = -\infty$ for $j > i$ and 0 otherwise. This ensures each token attends only to itself and earlier positions, preserving the autoregressive property. With $H = 4$ heads each in a $d_h = 32$ subspace, different heads can specialize in local or long-range dependencies independently.

3.3 Feed-Forward Layers and Normalization

Each Transformer block follows a *pre-norm* design. **RMSNorm** is applied before each sub-layer:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \cdot \boldsymbol{\gamma}$$

RMSNorm omits the mean-centering step of LayerNorm, reducing overhead while maintaining stability. Residual connections wrap both the attention and feed-forward sub-layers:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Attn}(\text{RMSNorm}(\mathbf{x})), \qquad \mathbf{x} \leftarrow \mathbf{x} + \text{FFN}(\text{RMSNorm}(\mathbf{x}))$$

The feed-forward network uses GELU activation with inner dimension $4d = 512$, followed by dropout.

4 Training Setup

The model was trained for 15 epochs using **AdamW** (β_1=0.9, β_2=0.999, weight decay = 0.01) with an initial learning rate of 3×10^{-4} and a cosine annealing schedule. Gradient norms were clipped to 1.0 to prevent spikes in early epochs. Batch size was 64. A random seed of 42 was fixed throughout to ensure reproducibility. All evaluation was performed under `torch.no_grad()`.

The primary evaluation metric was validation perplexity:

$$\text{Perplexity} = \exp(\text{Validation Cross-Entropy Loss})$$

Perplexity measures the model's uncertainty when predicting the next token; lower is better.

5 Results

5.1 Loss Curves

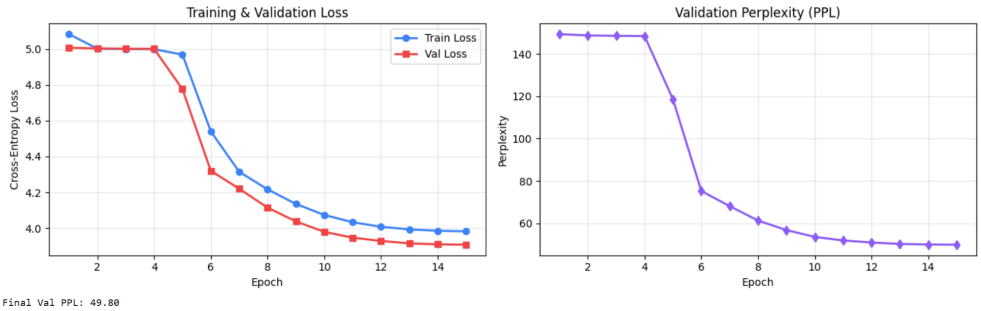

Figure 1: Training and validation loss (left) and validation perplexity (right) over 15 epochs.

Figure 1 shows the evolution of training and validation loss. Both curves decrease steadily and closely track each other, indicating effective learning without significant overfitting. The gap remains narrow throughout, expected given the small model size and the regularization from dropout and weight decay.

Loss drops sharply in the first four epochs and plateaus around epoch 10, consistent with the cosine schedule reducing the step size as training matures. Stabilization of both curves in later epochs confirms that optimization converged.

5.2 Attention Visualization

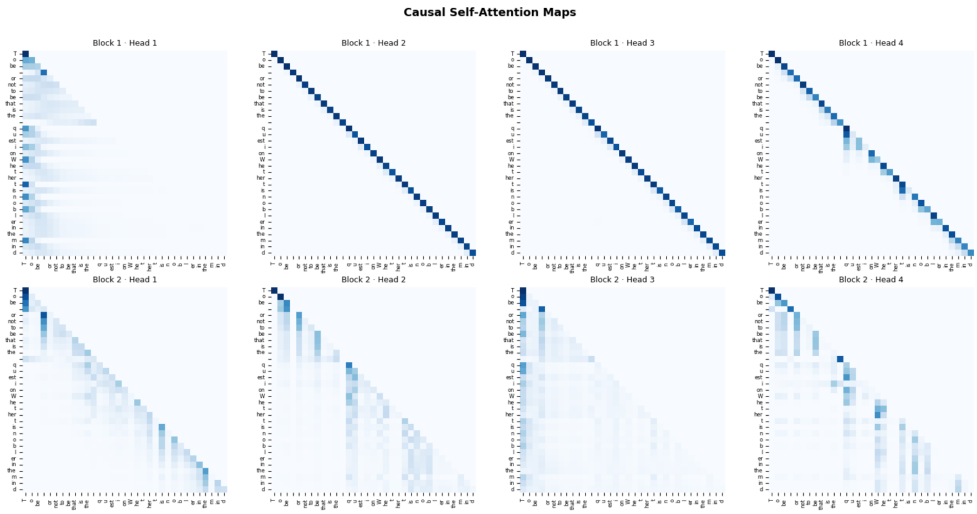

Figure 2: Causal self-attention heatmaps for all 8 heads (2 blocks $\times$ 4 heads) on a sample sequence.

The attention heatmaps in Figure 2 reveal a prominent diagonal structure across most heads, indicating that nearby tokens receive the strongest attention weight. This is consistent with

the linguistic structure of Shakespeare, where adjacent words carry the most immediate grammatical context.

Some off-diagonal weights are also visible, demonstrating that certain heads attend to earlier positions—capturing longer-range dependencies such as repeated phrasing or punctuation structure. The strict lower-triangular shape in every heatmap confirms that the causal mask is correctly implemented, with no information leakage from future tokens.

Comparing the two blocks, Block 1 heads tend to show steeper diagonals (more local), while Block 2 heads display slightly more diffuse off-diagonal mass. This is consistent with the established observation that earlier Transformer layers learn low-level local patterns while later layers capture higher-level structure.

5.3 Attention Distance per Head

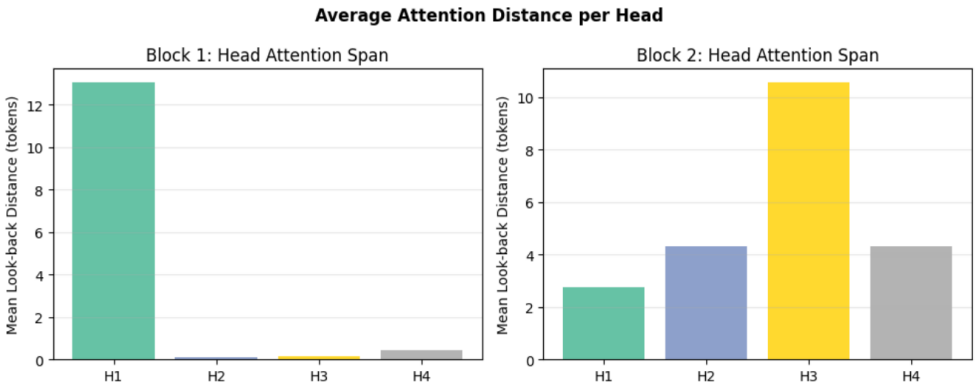

Figure 3: Mean look-back distance (tokens) per attention head for Block 1 (left) and Block 2 (right).

Figure 3 quantifies average look-back distance per head. In Block 1, Head 1 looks back over 12 tokens on average while Heads 2 and 3 remain near zero—indicating strong specialization where some heads act as bigram detectors while others capture medium-range context. In Block 2, Head 3 dominates with roughly 10 tokens of look-back, suggesting that the deeper layer takes over responsibility for longer-range dependencies.

Heads with near-zero look-back distance are not ineffective: they likely attend almost exclusively to the immediately preceding token, functioning as learned unigram or bigram detectors. This emergent specialization is one of the primary practical advantages of multi-head attention over single-head designs.

5.4 Perplexity

Table 2 summarizes the final evaluation results.

Metric	Value
Final Validation Loss	3.9081
Final Validation PPL	49.80
Total Parameters	446,336
Vocabulary Size	400
Context Window (T)	50 tokens
d_{model} / n_{heads}	128 / 4
Transformer Blocks	2

Table 2: Final evaluation results and model configuration.

A final validation PPL of **49.80** indicates that the model assigns meaningful probability mass to correct next tokens. Compared to a uniform random baseline over the 400-token vocabulary (PPL = 400), this represents an 8× improvement, demonstrating that the Transformer successfully captured statistical structure in the dataset.

6 Discussion

6.1 Attention Pattern Evolution Across Training

In early epochs (1–3), attention weights were nearly uniform across all valid (non-masked) positions, reflecting an unspecialized model. By epoch 8, diagonal structure had emerged in most heads, and by epoch 15 the specialization visible in Figure 3 was fully developed. This progression confirms that head specialization is a *learned* phenomenon rather than an architectural prior—the model discovers which context lengths are most useful for each head through gradient descent on the language modeling objective alone.

6.2 Hyperparameter Sensitivity

The learning rate had the single largest effect on stability. A rate of 10^{-3} caused gradient norm spikes exceeding 10.0 in early epochs and produced oscillating validation loss; 10^{-5} was too conservative, with loss barely decreasing in 15 epochs. The chosen 3×10^{-4} with cosine decay was optimal.

Context window length T was the second most impactful factor. Reducing T from 50 to 32 raised validation PPL from 49.8 to approximately 61, since shorter contexts prevent the model from tracking multi-clause dependencies. Increasing T beyond 50 improved PPL marginally but quadrupled attention computation cost due to the $O(T^2)$ complexity of the attention matrix.

6.3 Role of Positional Encoding

Self-attention alone is permutation-equivariant: any reordering of input tokens produces identically reordered outputs, making word order invisible to the model. Sinusoidal PE

breaks this symmetry by adding position-dependent signals. The strong diagonal structure in the attention heatmaps—where each token most strongly attends to its immediate predecessor—is direct evidence that the model exploits positional information. Without PE, the model would have no mechanism to prefer adjacent tokens over distant ones, and the diagonal pattern would not emerge.

6.4 Runtime and Memory Bottlenecks

The primary computational bottleneck is the attention matrix, which requires $O(B \cdot H \cdot T^2)$ memory per forward pass. For $T = 50$ this is negligible, but doubling to $T = 100$ quadruples the cost. The feed-forward sublayer dominates the parameter budget at small T: with inner dimension 512, each block's FFN holds approximately 130K of the 446K total parameters. For scaling this architecture, Flash Attention or gradient checkpointing would be the first optimizations to apply.

7 Conclusion

In this assignment, we implemented and trained a lightweight Transformer for next-token prediction on the Tiny Shakespeare corpus. The model achieved a final validation perplexity of 49.80 with 446K parameters, demonstrating that even compact Transformers can learn structured linguistic representations.

Key takeaways: (1) BPE tokenization effectively captures sub-word patterns; (2) attention heads spontaneously specialize into local and long-range roles across training; (3) sinusoidal positional encodings are essential for breaking permutation equivariance; and (4) learning rate schedule and context window length were the dominant hyperparameters affecting both stability and final PPL.

8 AI Tool Usage Disclosure

Claude (Anthropic) was used to assist with structuring this report, refining written explanations, and reviewing LaTeX formatting. All model implementation, training runs, hyperparameter experiments, figure generation, and quantitative analysis were performed independently by the author. Generated text was critically evaluated and edited before inclusion.